

Project 1

Business Mention Resolution Platform

Requirements Document

Data policy: Use the Yelp Open Dataset, or another publicly available local business dataset, as the source of data for this project. Do not scrape, collect, or use any private, internal, or personally identifiable data at any stage. Read and follow the dataset's usage terms before downloading it.

Overview

Build a system that reads free form text mentioning local businesses, such as reviews or short write ups, and resolves each mention to the correct business record in a catalog, even when multiple businesses share the same or a similar name. The system should extract business mentions from text, generate candidate matches, score how confident each match is, and route low confidence matches to a reviewer for manual confirmation rather than resolving them automatically. It should also include a smart assistant that can answer natural language questions about the catalog and complete a small multi step task on its own.

Requirements

1. Maintain records for businesses, business mentions, categories, and resolution results, including their details and status.
2. Support two types of business records: verified (added and confirmed by an admin) and unverified (submitted but not yet confirmed), which behave slightly differently when a mention is resolved against them. For example, a match against an unverified business should always be sent for review, even if the confidence score is high.
3. Store all data in a database with proper relationships (e.g. a category can have many businesses, and a business can have many mentions resolved to it).
4. Allow the system's data structure to be updated later without losing existing data (e.g. adding a new "business hours" record type).
5. Provide a way to create, view, update, and delete businesses, categories, and mentions.
6. Allow searching and filtering businesses by name, category, or city.
7. Show results in manageable pages rather than all at once.
8. Require users to log in, and support three types of users: admin (manages the catalog), reviewer (handles the review queue), and viewer (can query resolution results only).
9. Restrict certain actions, such as approving or rejecting a match or deleting a catalog entry, to admins and reviewers only.
10. Validate all user input carefully and protect against excessive or abusive requests, especially around the resolution endpoint.
11. Include tests that confirm the core logic works correctly, especially candidate matching and confidence threshold behaviour.
12. Generate a resolution summary document after a mention is successfully resolved.

13. Generate a monthly report document (mentions processed, match rate, most common review reasons).
14. Any generated document (resolution summary, monthly report) should be saved to disk and made available for download through the API, rather than returned directly in the response body.
15. Package the system so it can be run consistently on any machine.
16. Separate the system into two independent parts: one that manages business and catalog data and resolution logic, and one that handles notifications and documents, with the two communicating with each other.
17. Add a question answering feature: a user can ask a natural language question about the catalog (e.g. "which cafes in this area have the most mentions?") and get an answer based on the actual catalog data, referencing which records were used.
18. Add a smart assistant that can complete a small multi step task on its own. For example, given a new mention with two or more close scoring candidates, it should gather extra context from the surrounding text, decide on a match or escalate it for review, and draft a short note explaining the decision, without being told each step individually.

Suggested starting points

These are examples to start your own research from, not a required list. You are expected to look into other options and justify whatever you choose in your documentation.

- Backend API framework: for example, FastAPI, which you have already used.
- Database: for example, SQLite or PostgreSQL, accessed through an ORM such as SQLAlchemy.
- Mention or entity extraction: for example, spaCy's built in NER pipelines, a fine tuned BERT based token classifier, or a zero shot span extraction model such as GLiNER.
- Text embeddings for similarity: for example, sentence-transformers/all-MiniLM-L6-v2, BAAI/bge-base-en-v1.5, or intfloat/e5-base-v2.
- Vector storage: for example, FAISS, Chroma, or a self hosted Qdrant instance.

Bonus (optional)

This section is optional and outside the core requirements. Attempt it only after everything above is complete.

- Automatically refresh the list of items waiting in the review queue on a regular schedule, for example every night, so reviewers always start with an up to date queue. Example tools: APScheduler, or a simple scheduled script.